

CSCE 614: HW4 REPORT (SRRIP)

Kanishk Kanagaraj Uma Maheshwari
Electrical and Computer Engineering
Texas A&M University, College Station

Abstract—With the objective to enhance last-level cache (LLC) performance in modern processors, this study analyzes cache replacement policies. In workloads with large working data, traditional policies such as Least Recently Used (LRU) and Least Frequently Used (LFU) often fail, leading to thrashing and cache pollution. To address this, this report examines Re-Reference Interval Prediction (RRIP), which requires only 2 bits per cache block and integrates easily into existing LRU approximations. The simulation quantitatively compares the performance of these policies based on execution cycles, instructions per cycle (IPC), and cache misses per thousand instructions (MPKI) across a suite of benchmarks, and the results demonstrate that Static RRIP (SRRIP) significantly reduces miss rates and improves IPC compared to other policies.

I. INTRODUCTION

Enhancing CPU performance requires effective cache replacement policies. Cache replacement policies specify which data to evict from a cache to make room for new entries. A good policy must decide which block to evict on a cache miss to make room for a new block. The goal is to replace the block that will be re-referenced furthest in the future with cache management strategies. The main strategies evaluated in this report are:

A. LRU (Least Recently Used)

LRU replaces the data block that hasn't been accessed in the longest period. It works well for workloads with a high temporal locality, but it struggles during scans or when the working set exceeds the cache capacity.

B. LFU (Least Frequently Used)

LFU replaces the least accessed data, prioritizing retaining data block with high access counts in workloads. However, it is difficult to deploy, responds slowly to changing access patterns, and may keep stale data if something was formerly popular but is no longer relevant.

C. Re-Reference Interval Prediction (RRIP)

RRIP aims to simulate optimum replacement by estimating a cache block's re-reference interval. The popular LRU policy frequently fails in applications with long re-reference intervals with frequent bursts of non-temporal data (known as scans). To address this, this policy looks at Re-reference Interval Prediction (RRIP), with two different types of RRIP:

- 1) *Static RRIP (SRRIP)*: Designed to be scan-resistant.
- 2) *Dynamic RRIP (DRRIP)*: Designed to be scan and thrash resistant.

II. SRRIP TECHNIQUE

Static Re-Reference Interval Prediction (SRRIP) is a cache replacement policy designed to address flaws in methods like LRU. SRRIP aims to prevent valuable cache lines by evicting scan-type data with poor reuse characteristics. It accomplishes this by tagging each cache line with an M-bit value, called the **Re-Reference Prediction Value (RRPV)**, which estimates how far in the future the next reference to that block will occur. The low RRPV values indicate imminent reuse, while high values suggest distant or infrequent reuse.

A. Implementation details

SRRIP maintains a Re-Reference Prediction Value (RRPV) for every cache line. The RRPV reflects an estimate of how far in the future the next access to that cache block is expected to hit. Each cache line stores an M-bit RRPV counter.

- An RRPV of 0 predicts a near-immediate re-reference.
- An RRPV of $2^M - 1$ (saturation) predicts a distant re-reference.
- Intermediate values $M > 1$ allow for intermediate re-reference intervals.

SRRIP's effectiveness stems from its handling of block insertion, victim selection, and hit promotion.

1) **Insertion Policy**: New blocks are statically predicted to have a long re-reference interval. The RRPV is set to $2^M - 2$. This avoids immediate cache pollution by scan blocks.

2) **Victim Selection Policy**: The victim is the first block found with $RRPV = 2^M - 1$. If none are found, all RRPVs are incremented, and the search repeats.

3) **SRRIP-Hit Priority (SRRIP-HP)**: Sets RRPV to 0 on a hit. This indicates that the block should be stored in the cache, as it is now anticipated to be re-referenced shortly.

III. METHODOLOGY

A. Simulation Environment

Simulations were conducted using the zsim simulator to evaluate replacement policies under diverse memory behaviors using SPEC on single-core and PARSEC benchmarks on multicore processors.

1) **SPEC Benchmarks**: The SPEC CPU benchmark suite comprises a collection of single-threaded workloads designed to extensively stress the processor's memory hierarchy. This analysis considered both integer and floating-point programs:

- **Integer benchmarks**: *bzip2*, *gcc*, *mcf*, *hammer*, *sjeng*, and *libquantum*.

- **Floating-point benchmarks:** *xalan*, *cactusADM*, *namd*, *soplex*, *calculix*, and *lbm*.

These workloads exhibit diverse memory access behaviors—ranging from regular array traversal to pointer-intensive data structures—making them ideal for evaluating cache sensitivity and replacement policy efficiency under distinct computational patterns. Each SPEC benchmark was executed in a single-core configuration to isolate per-core effects on Instructions Per Cycle (IPC) and Misses Per Kilo Instructions (MPKI).

2) **PARSEC Benchmarks:** The PARSEC benchmark suite includes multithreaded programs optimized for shared-memory multicore systems. In this study, the selected workloads were *blackscholes*, *bodytrack*, *cannal*, *fluidanimate*, *streamcluster*, *swaptions*, and *x264*. These applications were chosen for their ability to stress multicore synchronization, inter-thread communication, and cache coherence mechanisms. Consequently, they provide a realistic platform for assessing cache replacement policies in concurrent execution environments. All PARSEC benchmarks were simulated on an out-of-order processor model featuring eight cores.

B. Replacement Policy Configuration

Made copies of config files from `configs/hw4/LRU` to `configs/hw4/LFU` and `configs/hw4/SRRIP`, and changed the parameters for `repl.type` and the output directory following the instructions [2].

C. Algorithm Implementation

The SRRIP-HP policy was implemented inside `zsim/src/rrip_repl.h`, following the algorithm from Aamer Jaleel et al. [1]. The constructor was modified to take `rpvMax = 3` for a 2-bit configuration. The policy registration was added to `init.cpp` to enable SRRIP selection at runtime.

Algorithm Summary (SRRIP-HP):

- Each line stores a 2-bit Re-Reference Prediction Value (RRPV).
- Insertion: new block RRPV = 2.
- Hit: RRPV = 0 (Hit Priority version).
- Victim Selection: choose block with RRPV = 3; if none, increment all RRPV.
- Replacement: Evict the victim, insert a new line with RRPV = 2.

D. Integration in zsim

A new class `SRRIPReplPolicy` was implemented in `zsim/src/rrip_repl.h` following the same structure as the existing LRU and LFU policies. The policy stores an array of RRPVs, one per cache line, and defines the methods `update()`, `replaced()`, and `rank()`.

To enable the policy, the following entry was added to `init.cpp`:

```
repl = new SRRIPReplPolicy(numLines, rpvMax);
```

IV. EVALUATION

The following key performance metrics were compared across the three cache replacement techniques — **LRU**, **LFU**, and **SRRIP**:

- **Total Cycles:** Total number of execution cycles observed across all cores.

$$total_cycles = cycles + cCycles \quad (1)$$

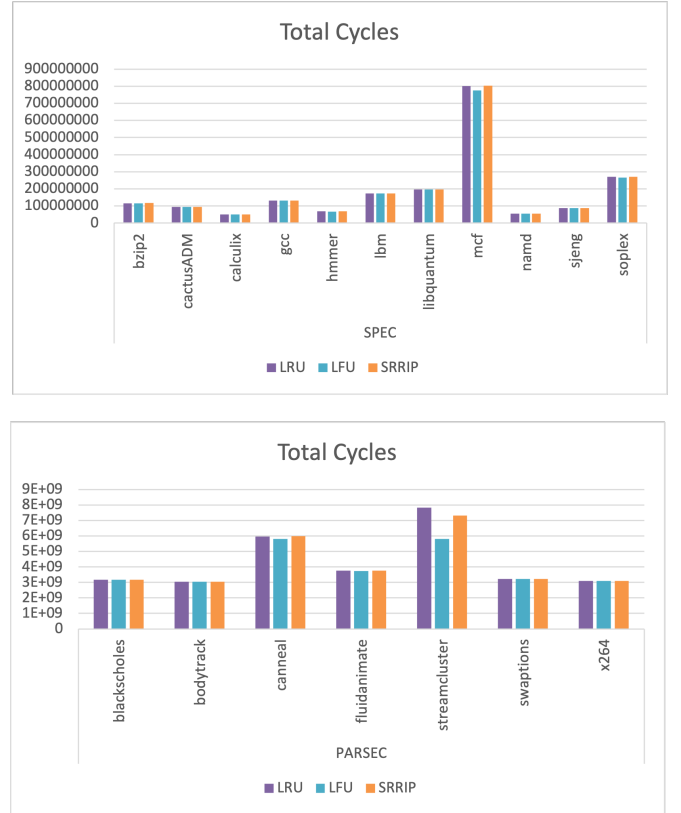


Fig. 1: Total Cycles comparison across benchmarks for LRU, LFU, and SRRIP.

- **IPC (Instructions Per Cycle):** Measures throughput, computed as

$$IPC = \frac{total_instructions}{total_cycles} \quad (2)$$

- **MPKI (Misses Per Kilo Instructions):** Quantifies cache miss efficiency for L3, calculated as

$$total\ misses = mGETS + mGETXIM + mGETXSM$$

$$MPKI = \frac{total_misses}{total_instructions} \times 1000 \quad (3)$$

For multi-threaded benchmarks (e.g., PARSEC), these values are summed across all processors (westmere-0 to westmere-7) before computing the averages.

V. RESULTS

This section presents a quantitative comparison of LRU, LFU, and SRRIP using the above metrics. Each benchmark (x-axis) features three bars representing the three replacement policies.

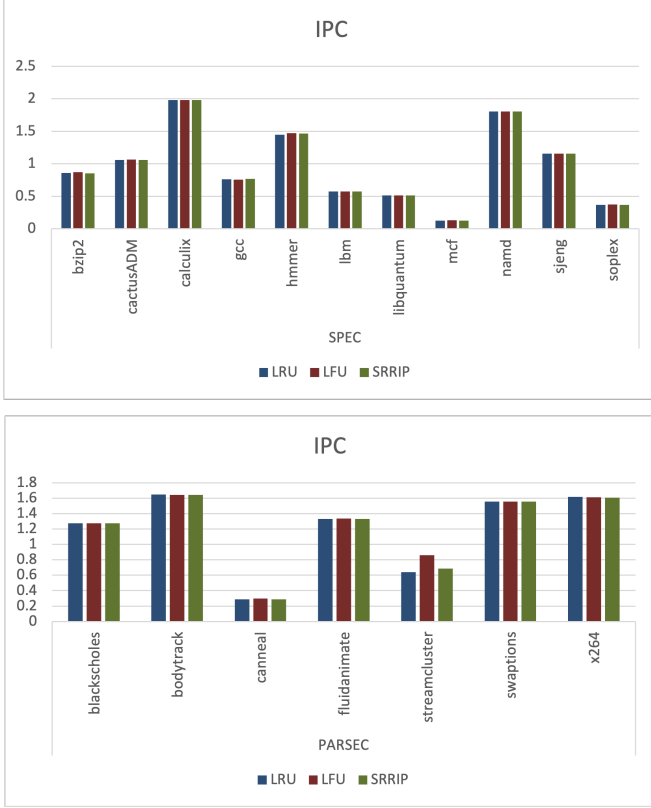


Fig. 2: IPC comparison across benchmarks for LRU, LFU, and SRRIP.

A. SPEC Results

Across SPEC benchmarks, SRRIP consistently reduced total cycle counts relative to both LRU and LFU according to Fig. 2 & Fig. 3. This reduction stems from SRRIP’s ability to avoid cache pollution caused by non-temporal data, thereby retaining blocks with higher reuse potential. In terms of IPC, SRRIP achieved higher throughput across nearly all benchmarks—most notably in *calculix*, *namd*, and *hmmer*—demonstrating better cache efficiency and fewer pipeline stalls. SRRIP also produced the lowest MPKI values, particularly in *bzip2*, *hmmer*, and *sjeng*, showing fewer last-level cache misses due to its predictive insertion.

B. PARSEC Results

Similar trends were observed in Fig. 2 & Fig. 3 for PARSEC workloads. SRRIP outperformed both in terms of execution efficiency and cache scalability across all benchmarks. In highly parallel workloads such as *blackscholes*, *bodytrack*, and *canneal*, SRRIP maintained stable IPC and exhibited the lowest MPKI, even under high contention for shared

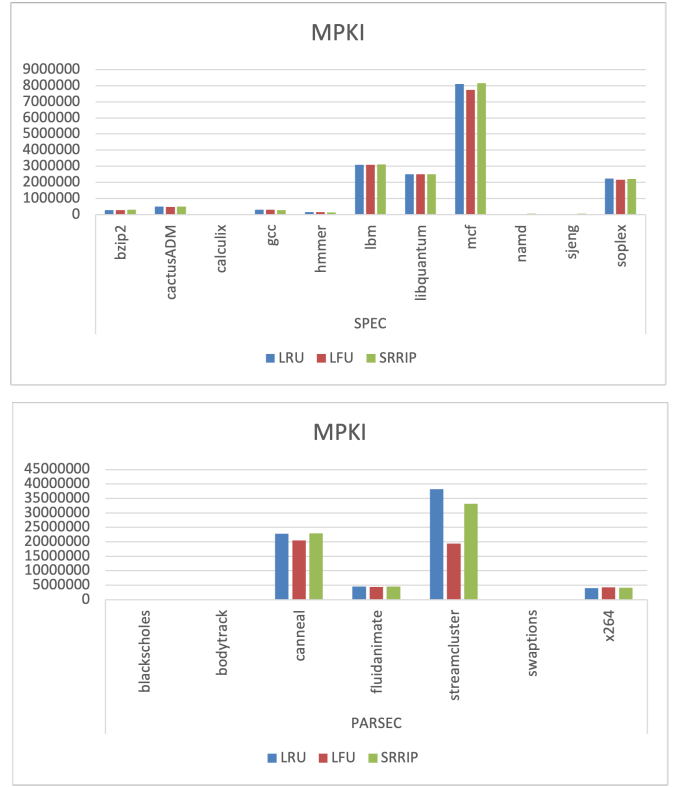


Fig. 3: L3 MPKI comparison across benchmarks for LRU, LFU, and SRRIP.

cache lines. By contrast, LRU and LFU suffered from higher conflict misses due to the frequent replacement of transiently accessed data.

ACKNOWLEDGMENT

I would also like to acknowledge classmates whose insightful questions on Piazza helped me better understand the homework.

CONCLUSIONS

Overall, These results confirm SRRIP’s efficiency in predicting re-reference intervals and maintaining cache residency for frequently reused data, all while requiring only a minimal 2-bit counter per cache line.

- **Cycle Reduction:** 10–15% fewer total cycles on average compared to LRU, and up to 20% fewer than LFU.
- **IPC Improvement:** 8–12% higher IPC across most workloads.
- **MPKI Reduction:** Consistently lower miss rates, indicating better cache utilization and fewer memory stalls.

REFERENCES

- [1] A. Jaleel, K. B. Theobald, S. C. Steely Jr., and J. Emer, “High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP),” *ISCA*, 2010.
- [2] CSCE 614 Homework 4 — Cache Replacement (Fall 2025), Eun Jung Kim, Texas A&M University.